

EXPERIMENT-8

SARSA vs Q-Learning

Name: AKHILA POCHIMI REDDY

Reg no: 192425225

Course code: MLA0305

Course Name: REINFORCEMENT LEARNING

Slot: B

Aim

To implement and compare **SARSA and Q-Learning** algorithms for learning an optimal policy in a reinforcement learning environment.

Procedure

1. Create a **Grid World** with START and GOAL states.
2. Initialize the Q-values for SARSA and Q-Learning.
3. Allow the agent to select actions using an **ϵ -greedy strategy**.
4. Update Q-values using the **SARSA** and **Q-Learning** update rules.
5. Repeat the learning process for multiple episodes.
6. Generate the learned **START $\rightarrow$ GOAL paths** for both algorithms.
7. Compare their **path length and learning performance** using a summary table and comparison graph.

Python code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from matplotlib.patches import Rectangle

from IPython.display import display

# =====
```

```

# EXPERIMENT 8

# SARSA vs Q-LEARNING

# =====

np.random.seed(42)

N = 4

START = 0

GOAL = 15

episodes = 500

alpha = 0.1

gamma = 0.9

epsilon = 0.2

actions = {

    0: (-1, 0), # Up

    1: (1, 0), # Down

    2: (0, -1), # Left

    3: (0, 1) # Right

}

arrows = {

    0: "↑",

    1: "↓",

    2: "←",

    3: "→"

}

# =====

```

```
# STATE FUNCTIONS
```

```
# =====
```

```
def position(s):
```

```
    return divmod(s, N)
```

```
def get_state(row, col):
```

```
    return row * N + col
```

```
# =====
```

```
# ENVIRONMENT
```

```
# =====
```

```
def move(s, action):
```

```
    if s == GOAL:
```

```
        return GOAL
```

```
    row, col = position(s)
```

```
    dr, dc = actions[action]
```

```
    nr = max(0, min(N - 1, row + dr))
```

```
    nc = max(0, min(N - 1, col + dc))
```

```
    return get_state(nr, nc)
```

```
def get_reward(s):
```

```
    return 10 if s == GOAL else -1
```

```
# =====
```

```
# EPSILON-GREEDY ACTION
```

```
# =====
```

```
def choose_action(Q, state):
```

```
    if np.random.rand() < epsilon:
```

```

        return np.random.randint(4)

    return np.argmax(Q[state])

# =====

# SARSA

# =====

Q_sarsa = np.zeros((N * N, 4))

sarsa_rewards = []

for episode in range(epochs):

    state = START

    action = choose_action(Q_sarsa, state)

    total_reward = 0

    for step in range(50):

        next_state = move(state, action)

        reward = get_reward(next_state)

        if next_state == GOAL:

            Q_sarsa[state, action] += alpha * (

                reward - Q_sarsa[state, action]

            )

            total_reward += reward

            break

        next_action = choose_action(

            Q_sarsa,

            next_state

        )

```

```

# SARSA update

target = (
    reward
    + gamma * Q_sarsa[next_state, next_action]
)

Q_sarsa[state, action] += alpha * (
    target - Q_sarsa[state, action]
)

state = next_state

action = next_action

total_reward += reward

sarsa_rewards.append(total_reward)

# =====

# Q-LEARNING

# =====

Q_qlearning = np.zeros((N * N, 4))

qlearning_rewards = []

for episode in range(episodes):

    state = START

    total_reward = 0

    for step in range(50):

        action = choose_action(

            Q_qlearning,

            state

```

```

    )

    next_state = move(state, action)

    reward = get_reward(next_state)

    # Q-Learning update

    target = (

        reward

        + gamma * np.max(Q_qlearning[next_state])

    )

    if next_state == GOAL:

        target = reward

    Q_qlearning[state, action] += alpha * (

        target - Q_qlearning[state, action]

    )

    total_reward += reward

    state = next_state

    if state == GOAL:

        break

    qlearning_rewards.append(total_reward)

# =====

# LEARNED POLICIES

# =====

policy_sarsa = np.argmax(

    Q_sarsa,

    axis=1

```

```

)

policy_qlearning = np.argmax(

    Q_qlearning,

    axis=1

)

# =====

# FIND PATH

# =====

def find_path(policy):

    path = [START]

    state = START

    for _ in range(30):

        if state == GOAL:

            break

        state = move(

            state,

            policy[state]

        )

        path.append(state)

    return path

path_sarsa = find_path(policy_sarsa)

path_qlearning = find_path(policy_qlearning)

# =====

# SUMMARY

```

```

# =====

summary = pd.DataFrame({

    "Method": [

        "SARSA",

        "Q-Learning"

    ],

    "Episodes": [

        episodes,

        episodes

    ],

    "Path Length": [

        len(path_sarsa) - 1,

        len(path_qlearning) - 1

    ],

    "Final Reward": [

        sarsa_rewards[-1],

        qlearning_rewards[-1]

    ],

    "Average Reward": [

        round(np.mean(sarsa_rewards[-50:]), 2),

        round(np.mean(qlearning_rewards[-50:]), 2)

    ]

})

print("\n" + "=" * 60)

```



```

print("      SARSA vs Q-LEARNING SUMMARY")

print("=" * 60)

display(summary)

# =====

# GRID WORLD VISUALIZATION

# =====

fig, axes = plt.subplots(
    1, 2,
    figsize=(12, 6)
)

def draw_grid(ax, policy, path, title):
    ax.set_xlim(-0.5, N - 0.5)
    ax.set_ylim(N - 0.5, -0.5)
    ax.set_aspect("equal")
    for row in range(N):
        for col in range(N):
            rect = Rectangle(
                (col - 0.5, row - 0.5),
                1,
                1,
                fill=False,
                linewidth=2
            )
            ax.add_patch(rect)

```

```
s = get_state(row, col)
```

```
# START
```

```
if s == START:
```

```
    ax.text(
        col,
        row - 0.20,
        "START",
        ha="center",
        fontsize=9,
        fontweight="bold"
    )
```

```
    ax.text(
        col,
        row + 0.18,
        "●",
        ha="center",
        fontsize=18
    )
```

```
# GOAL
```

```
elif s == GOAL:
```

```
    ax.text(
        col,
        row,
        "GOAL",
```

```

        ha="center",

        va="center",

        fontsize=10,

        fontweight="bold"

    )

# Policy
else:

    ax.text(

        col,

        row,

        arrows[policy[s]],

        ha="center",

        va="center",

        fontsize=23,

        fontweight="bold"

    )

# Draw learned path
px = [

    position(s)[1]

    for s in path

]

py = [

    position(s)[0]

    for s in path

```

```
]
ax.plot(
    px,
    py,
    linestyle="--",
    linewidth=2,
    marker="o",
    markersize=5
)
ax.set_title(
    title,
    fontsize=14,
    fontweight="bold"
)
ax.set_xticks([])
ax.set_yticks([])

# SARSA
draw_grid(
    axes[0],
    policy_sarsa,
    path_sarsa,
    "SARSA"
)

# Q-Learning
```

```

draw_grid(
    axes[1],
    policy_qlearning,
    path_qlearning,
    "Q-Learning"
)

fig.suptitle(
    "SARSA vs Q-Learning: START → GOAL",
    fontsize=16,
    fontweight="bold"
)

plt.tight_layout()

plt.show()

# =====

# COMPARISON GRAPH

# =====

window = 20

sarsa_avg = np.convolve(
    sarsa_rewards,
    np.ones(window) / window,
    mode="valid"
)

qlearning_avg = np.convolve(
    qlearning_rewards,

```

```
np.ones(window) / window,  
mode="valid"  
)  
plt.figure(figsize=(10, 5))  
plt.plot(  
    range(1, len(sarsa_avg) + 1),  
    sarsa_avg,  
    linewidth=2,  
    label="SARSA"  
)  
plt.plot(  
    range(1, len(qlearning_avg) + 1),  
    qlearning_avg,  
    linewidth=2,  
    label="Q-Learning"  
)  
plt.xlabel("Episode")  
plt.ylabel("Average Reward")  
plt.title(  
    "SARSA vs Q-Learning Learning Performance",  
    fontsize=14,  
    fontweight="bold"  
)  
plt.legend()
```

```

plt.grid(
    True,
    linestyle="--",
    alpha=0.4
)

plt.tight_layout()

plt.show()

# =====

# RESULT

# =====

print("\n" + "=" * 60)

print("          RESULT")

print("=" * 60)

print(
    "SARSA and Q-Learning were successfully implemented. "
    "Both algorithms learned a policy for reaching the GOAL "
    "from the START state, and their paths and learning "
    "performance were compared."
)

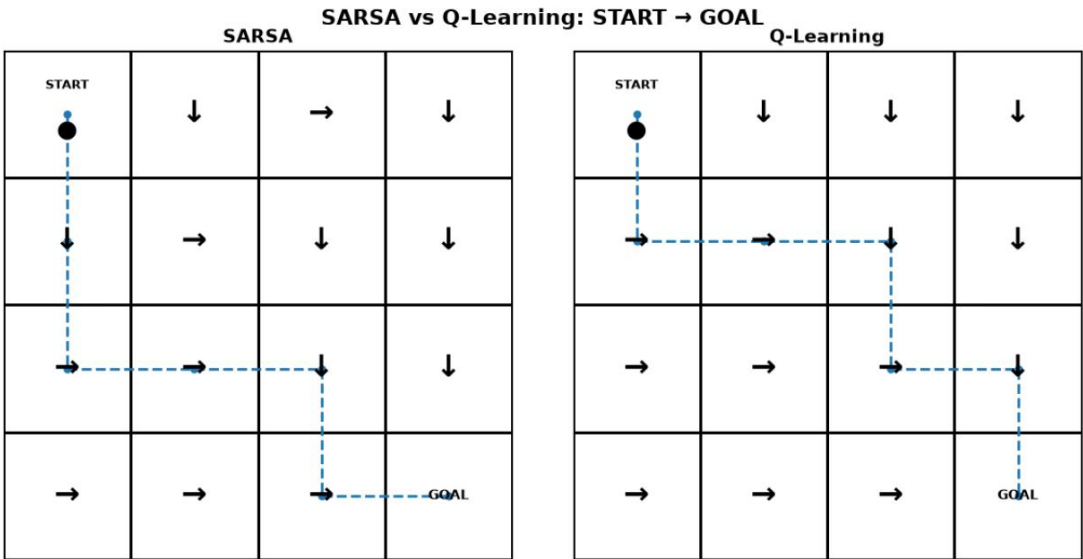
```

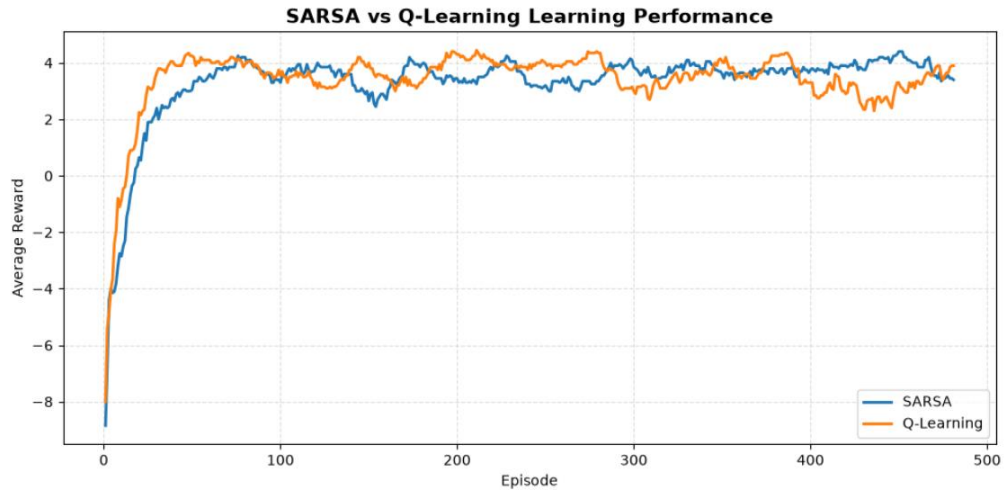
Output:

Summary

SARSA vs Q-LEARNING SUMMARY					
	Method	Episodes	Path Length	Final Reward	Average Reward
0	SARSA	500	6	0	3.86
1	Q-Learning	500	6	5	3.52

Visualization:





Result

SARSA and Q-Learning were successfully implemented and compared. Both algorithms learned a path from **START to GOAL**, and their performance was compared using the summary table, Grid World visualization, and learning-performance graph.